
Árvores Balanceadas

Red-Black Trees & B-Trees

Disciplina: Estrutura de Dados

Entrega: 15 de junho de 2026

Árvore Vermelho-Preto

Fundamentação teórica

Uma Árvore Vermelho-Preto (Red-Black Tree) é uma Árvore Binária de Busca que atribui uma cor — vermelho ou preto — a cada nó e mantém cinco propriedades que, em conjunto, impedem que a árvore fique desbalanceada. O resultado prático é que a altura nunca ultrapassa $2 \cdot \log_2(n+1)$, garantindo $O(\log n)$ para busca, inserção e remoção mesmo no pior caso — o que não acontece numa ABB simples, que pode degenerar para $O(n)$.

As cinco propriedades são: **(1)** todo nó é vermelho ou preto; **(2)** a raiz é sempre preta; **(3)** folhas sentinelas (NIL) são pretas; **(4)** um nó vermelho nunca tem filho vermelho; e **(5)** todo caminho de um nó até qualquer folha NIL descendente contém o mesmo número de nós pretos — chamado de *black-height*. É a propriedade 5 que matematicamente limita a altura da árvore.

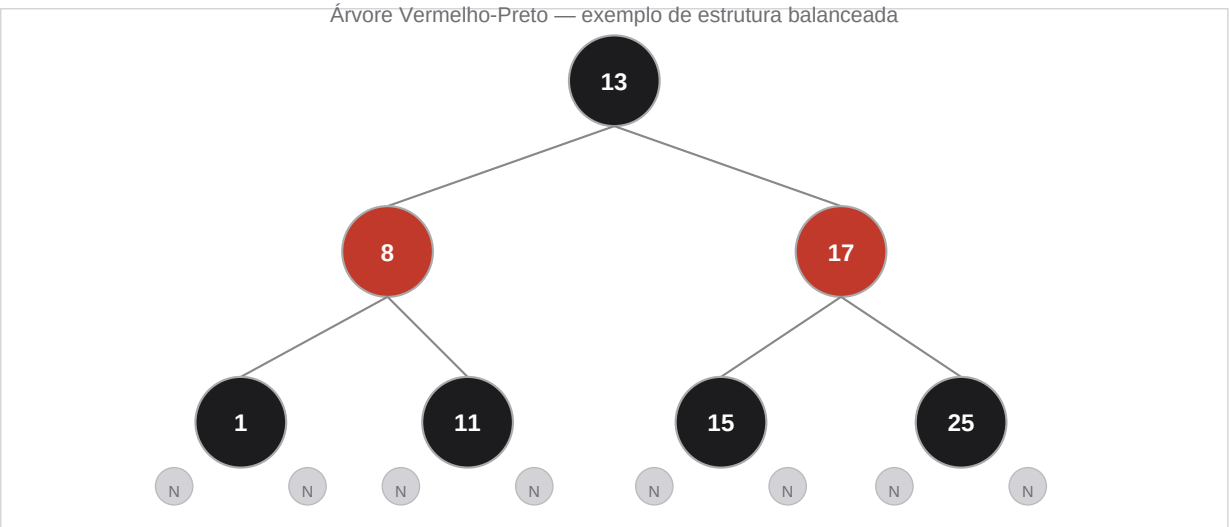


Figura 1 — Estrutura básica. Nós pretos e vermelhos alternados; folhas NIL (cinza) encerram todos os ramos.

Complexidade

Operação	Pior caso	Caso médio
Busca	$O(\log n)$	$O(\log n)$
Inserção	$O(\log n)$	$O(\log n)$
Remoção	$O(\log n)$	$O(\log n)$
Espaço	$O(n)$	$O(n)$

Operações

Busca

A busca é idêntica à de uma ABB comum: começa na raiz, desce à esquerda se a chave buscada for menor, à direita se for maior, e termina ao encontrar a chave ou atingir uma folha NIL. A coloração dos nós não é consultada — ela existe apenas para orientar o rebalanceamento nas operações de escrita.

```
SEARCH(T, k):  
  x ← T.raiz  
  ENQUANTO x ≠ NIL E k ≠ x.chave:  
    SE k < x.chave: x ← x.esq  
    SENÃO: x ← x.dir  
  RETORNAR x
```

Inserção e rebalanceamento

Todo nó novo é inserido como **vermelho** — isso preserva a black-height, mas pode violar a propriedade 4 (dois vermelhos consecutivos). A correção depende da cor do *tio* do nó inserido e segue três casos, ilustrados abaixo.

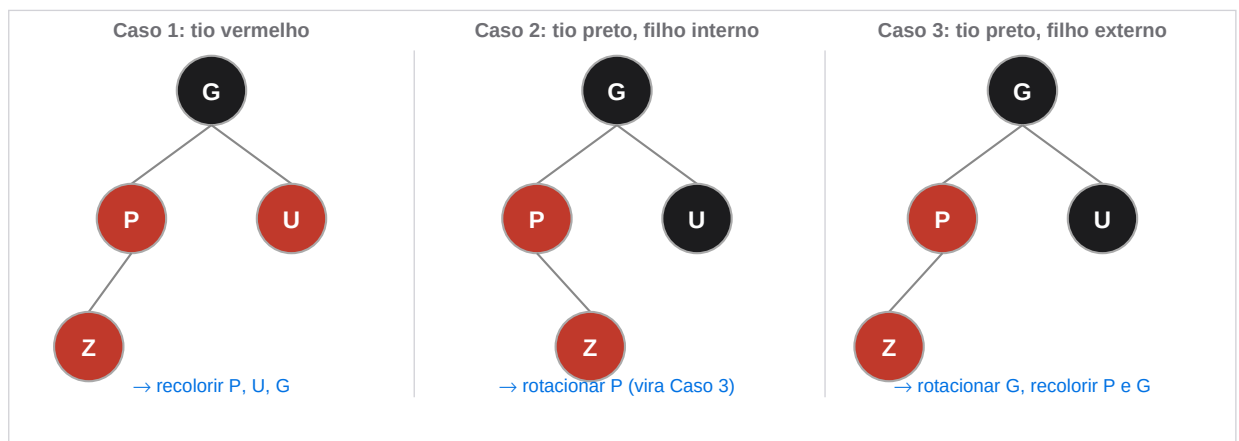


Figura 2 — Os três casos de violação após inserção e a ação corretiva de cada um.

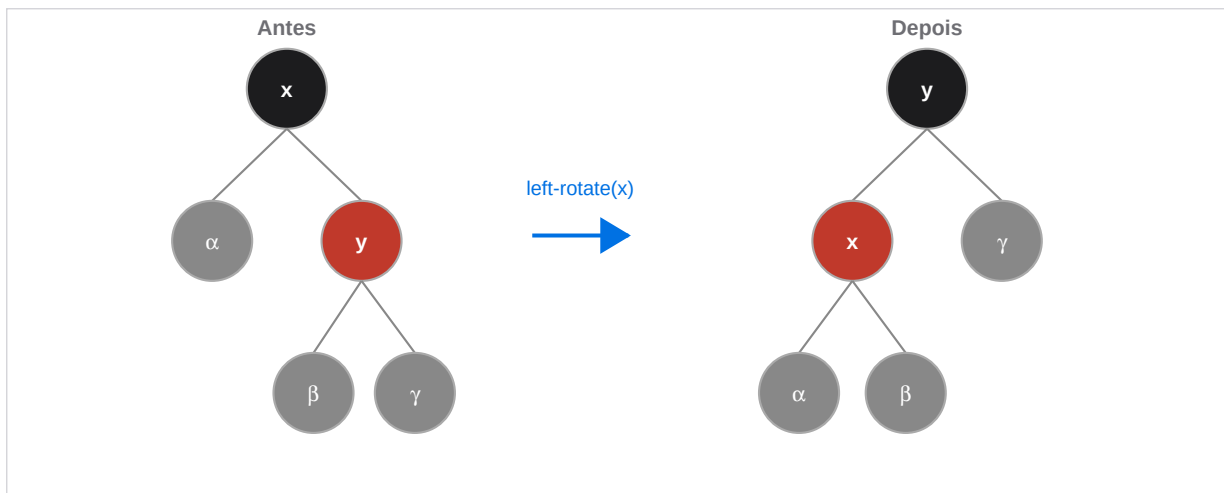


Figura 3 — Rotação à esquerda: reestrutura os ponteiros sem alterar a ordem das chaves.

No Caso 1, apenas recolorir o pai, o tio e o avô resolve o problema — mas pode propagar a violação para cima. Os Casos 2 e 3 envolvem pelo menos uma rotação e resolvem a violação localmente. No total, uma inserção realiza no máximo **2 rotações**.

Remoção

A remoção segue o algoritmo padrão de ABB (substituindo nós com dois filhos pelo sucessor in-order) e depois verifica se a black-height foi comprometida. Quando o nó removido ou seu substituto era preto, aplica-se o **RB-DELETE-FIXUP**, que trata quatro casos simétricos com recolorações e rotações. No pior caso são realizadas **3 rotações**.

Aplicações reais

Linux Kernel — CFS Scheduler e gerenciamento de memória

O kernel Linux mantém uma implementação genérica de Red-Black Tree em *lib/rbtree.c*, usada em vários subsistemas. O mais conhecido é o **Completely Fair Scheduler (CFS)**: cada processo executável é armazenado na RBT ordenado pelo seu *vruntime* (tempo virtual de CPU acumulado). O próximo processo a executar é sempre o nó mais à esquerda — mínimo da árvore —, acessível em $O(1)$ com cache. Inserções e remoções ocorrem em $O(\log n)$ quando um processo entra ou sai da fila.

O mesmo arquivo é usado pelo subsistema de memória virtual (*mm/mmap.c*) para indexar as VMAs (Virtual Memory Areas) de cada processo por endereço, permitindo localização rápida durante page faults e chamadas a *mmap/munmap*.

A escolha pela RBT — e não por uma Árvore B — é direta: tudo acontece em RAM. Não há custo de I/O em disco, então o alto fator de ramificação da B-Tree não traz vantagem; ao contrário, aumentaria o overhead por operação. A RBT binária ocupa apenas *key + cor + 3* ponteiros por nó e oferece latência previsível, essencial para um escalonador de tempo real.

Java Standard Library — TreeMap e TreeSet

O **java.util.TreeMap** do OpenJDK é implementado explicitamente sobre uma Red-Black Tree (descrito no próprio Javadoc como *"A Red-Black tree based NavigableMap implementation"*). O **TreeSet** usa internamente um **TreeMap**. Isso garante iteração em ordem, operações de intervalo (`subMap`, `headMap`, `tailMap`) e acesso ao mínimo e máximo, tudo em $O(\log n)$. A mesma abordagem é adotada pelo C++ STL (`std::map`, `std::set`) e pelo .NET `SortedDictionary`.

Árvore B

Fundamentação teórica

A Árvore B foi proposta por Bayer e McCreight em 1972 para resolver um problema específico: como organizar dados em disco minimizando o número de leituras de página. A ideia central é simples — em vez de armazenar uma única chave por nó como nas árvores binárias, cada nó da Árvore B armazena *muitas* chaves, dimensionado para ocupar exatamente uma página de disco (tipicamente 4 KB a 16 KB). Isso reduz drasticamente a altura da árvore e, por consequência, o número de leituras de I/O por operação.

Formalmente, uma Árvore B de **ordem t** (grau mínimo $t \geq 2$) obedece às seguintes regras: todo nó não-raiz possui entre $t-1$ e $2t-1$ chaves; um nó com k chaves tem exatamente $k+1$ filhos; todas as folhas estão na mesma profundidade; e as chaves dentro de cada nó estão ordenadas, servindo como separadores dos intervalos dos filhos. A altura satisfaz $h \leq \log_t((n+1)/2)$ — com $t=500$ e um bilhão de registros, isso resulta em **apenas 3 níveis**.

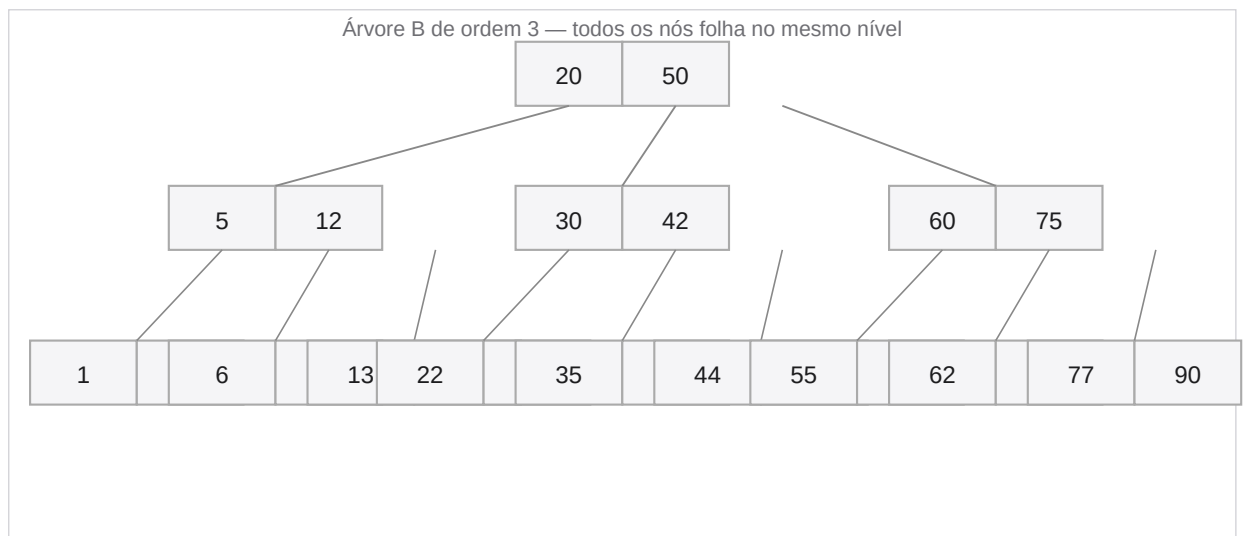


Figura 4 — Árvore B de ordem 3. Todos os nós folha estão no mesmo nível; cada nó agrupa múltiplas chaves.

Complexidade

Operação	Pior caso	Obs.
Busca	$O(t \cdot \log_t n)$	~3 leituras de disco para $n=10^9$, $t=500$
Inserção	$O(t \cdot \log_t n)$	No máximo $\log_t n$ splits
Remoção	$O(t \cdot \log_t n)$	No máximo $\log_t n$ merges

Espaço	$O(n)$	Aproveitamento mínimo de 50% por nó
--------	--------	-------------------------------------

Operações

Busca

A busca desce a árvore nível a nível. Em cada nó, realiza-se uma busca sequencial (ou binária, para nós grandes) pelas chaves para determinar em qual intervalo a chave buscada se encaixa e, portanto, qual ponteiro filho seguir. O passo crítico é o **DISK-READ**: cada nó visitado corresponde a uma leitura de página de disco.

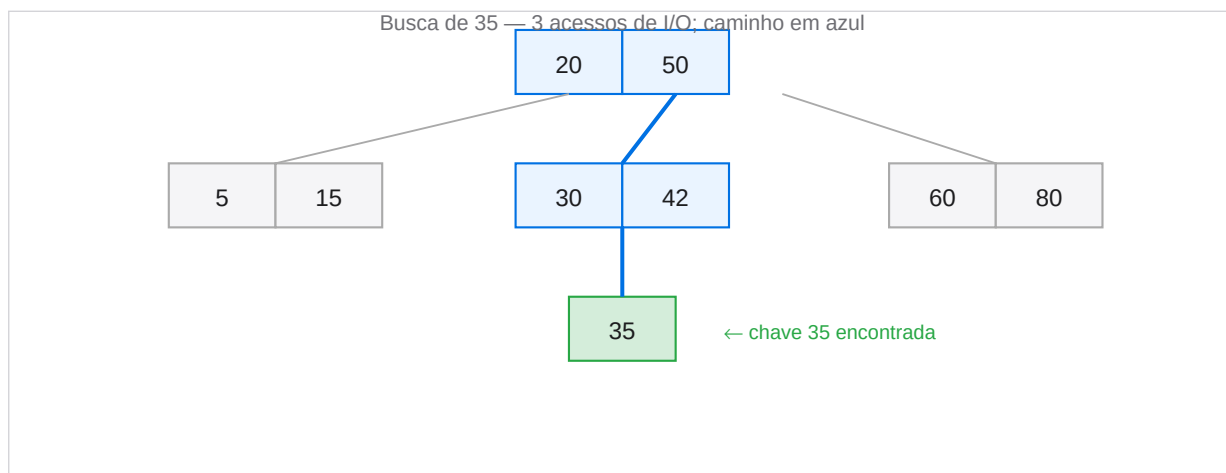


Figura 5 — Busca de 35: apenas 3 nós são acessados, independentemente do tamanho total da árvore.

```

B-TREE-SEARCH(x, k):
  i ← 1
  ENQUANTO i ≤ x.n E k > x.chave[i]: i ← i+1
  SE i ≤ x.n E k = x.chave[i]: RETORNAR (x, i)
  SE x.folha: RETORNAR NIL
  DISK-READ(x.filho[i])
  RETORNAR B-TREE-SEARCH(x.filho[i], k)
  
```

Inserção e split de nós

Inserções sempre ocorrem nas folhas. Ao descer pela árvore, qualquer nó cheio (com $2t-1$ chaves) é imediatamente dividido antes de prosseguir — isso garante que nunca seja necessário subir na árvore após a inserção, economizando leituras de disco.

O **split** de um nó cheio funciona assim: a chave mediana (posição t) sobe para o nó pai; as $t-1$ chaves menores ficam no filho esquerdo; as $t-1$ chaves maiores formam um novo filho direito.

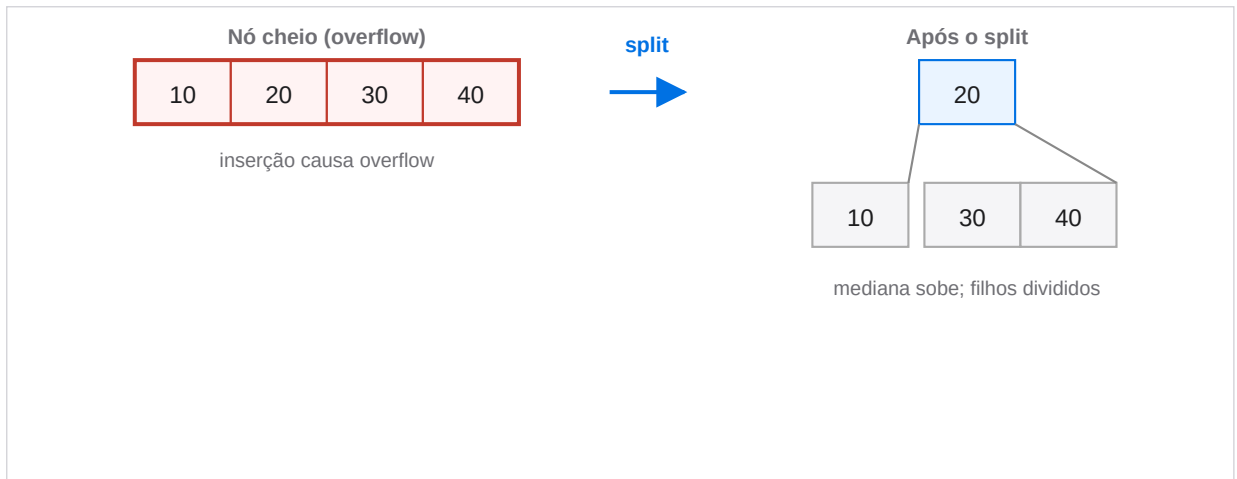


Figura 6 — Split: a mediana sobe para o pai e o nó se divide em dois. A árvore cresce em altura apenas pela raiz.

Remoção e merge de nós

A remoção é simétrica. Ao descer, qualquer nó com apenas $t-1$ chaves (o mínimo) é reforçado antes de prosseguir, por uma de duas estratégias: **rotação** (emprestar uma chave de um irmão adjacente via o pai, se ele tiver chaves sobrando) ou **merge** (fundir o nó atual com seu irmão e a chave separadora do pai, formando um nó com $2t-1$ chaves). O merge é o inverso exato do split. A remoção de uma chave interna é feita substituindo-a pelo predecessor ou sucessor in-order da folha.

Aplicações reais

PostgreSQL — índices B-Tree como padrão

O PostgreSQL usa Árvore B como **método de acesso padrão para índices**, implementado em *src/backend/access/nbtree/*. Cada nó corresponde a uma página de 8 KB, podendo armazenar centenas de chaves. Para uma tabela com bilhões de linhas, a árvore tipicamente tem entre 2 e 4 níveis — o que significa que qualquer registro é encontrado em no máximo 4 leituras de disco.

O PostgreSQL implementa especificamente a variante **B-link tree** de Lehman e Yao (1981), que adiciona ponteiros laterais entre nós do mesmo nível para permitir buscas e inserções concorrentes com locks de grão fino, sem bloquear leitores durante splits. A documentação oficial (postgresql.org/docs/current/btree.html) detalha esse design.

Um índice B-Tree em PostgreSQL suporta nativamente igualdade, comparações ($<$, $>$, BETWEEN) e ordenação (ORDER BY) — tudo com o mesmo índice, sem estruturas auxiliares.

Sistemas de arquivos — NTFS, ext4 e Btrfs

A Árvore B (e sua variante B+) aparece no coração de sistemas de arquivos modernos pelos mesmos motivos que em bancos de dados: dados em disco, acesso por chave, leituras de I/O caras. O **NTFS** indexa as entradas da Master File Table (MFT) em uma B+ Tree, permitindo

localizar qualquer arquivo em um diretório com milhões de entradas em pouquíssimas leituras. O **ext4** usa HTrees (baseadas em B-Tree) para indexar entradas de diretório com blocos de 4 KB. O **Btrfs** vai além: usa Árvores B para *tudo* — arquivos, checksums, snapshots e subvolumes — tornando a estrutura o núcleo arquitetural do sistema inteiro.

Análise Comparativa

Quadro comparativo

Critério	Árvore Vermelho-Preto	Árvore B
Ambiente natural	Memória RAM	Disco / SSD
Filhos por nó	2 (binária)	$t-1$ a $2t-1$ filhos
Altura máxima	$2 \cdot \log_2(n+1)$	$\log_{t+1}((n+1)/2)$
Leituras de disco ($n=10^6$, $t=500$)	~30 acessos	~3 acessos
Rebalanceamento	Rotação + recolorir	Split / Merge
Overhead por nó	Mínimo (5 campos)	Maior (vetor de chaves+filhos)
Range queries	Eficiente em RAM	Ótimo (folhas encadeadas na B+)
Tempo real / latência	Previsível (≤ 3 rotações)	Depende do I/O
Exemplos	Linux CFS, Java TreeMap, C++ map	PostgreSQL, InnoDB, NTFS, ext4

Quando usar cada estrutura

Se...	Prefira
Dados vivem em RAM	Árvore Vermelho-Preto
Dados em disco / banco de dados	Árvore B
Range queries frequentes (SQL)	Árvore B+
Escalonador / sistema de tempo real	Árvore Vermelho-Preto
Índice em filesystem	Árvore B
Coleção padrão em linguagem (map/set)	Árvore Vermelho-Preto

O critério decisivo: onde os dados vivem

A pergunta mais importante ao escolher entre as duas estruturas não é sobre complexidade assintótica — ambas são $O(\log n)$. É sobre **onde os dados residem**.

Quando os dados estão em RAM, acessos a ponteiros custam nanosegundos. A estrutura binária da RBT é ideal: overhead mínimo por nó, operações previsíveis e no máximo 3 rotações por operação. É por isso que o escalonador do Linux, o Java TreeMap e o C++ `std::map` usam RBT — todos operam inteiramente em memória.

Quando os dados estão em disco ou SSD, a equação muda radicalmente. Uma leitura de página custa microssegundos a milissegundos — **100.000× mais lento** que a RAM. Cada nível a mais na árvore significa mais uma leitura de disco. A Árvore B resolve isso aumentando o fator de ramificação para centenas ou milhares, reduzindo a altura a 2–4 níveis independentemente do tamanho do dataset. É por isso que todo banco de dados relacional sério — PostgreSQL, MySQL, Oracle, SQLite — usa B-Tree para seus índices.

Não é coincidência que o Linux use **RBT para o scheduler em memória** e **Btrfs use B-Tree para o filesystem em disco**. São escolhas conscientes, cada uma otimizada para o ambiente onde vai operar. Entender essa distinção é entender por que estruturas de dados não existem em abstrato — elas existem em contexto.

REFERÊNCIAS

- [1] CORMEN, T. H. et al. *Introduction to Algorithms*. 4. ed. MIT Press, 2022. Caps. 13 e 18.
- [2] BAYER, R.; MCCREIGHT, E. Organization and Maintenance of Large Ordered Indices. *Acta Informatica*, v. 1, p. 173–189, 1972.
- [3] GUIBAS, L.; SEDGEWICK, R. A Dichromatic Framework for Balanced Trees. *19th Annual Symposium on FOCS*, 1978.
- [4] TORVALDS, L. et al. *Linux Kernel* — `lib/rbtree.c`. github.com/torvalds/linux/blob/master/lib/rbtree.c
- [5] TORVALDS, L. et al. *Linux Kernel* — `kernel/sched/fair.c`. github.com/torvalds/linux/blob/master/kernel/sched/fair.c
- [6] ORACLE. *Java SE 21* — `java.util.TreeMap`. docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeMap.html
- [7] POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Docs* — *B-Tree Indexes*. postgresql.org/docs/current/btree.html
- [8] LEHMAN, P. L.; YAO, S. B. Efficient Locking for Concurrent Operations on B-Trees. *ACM TODS*, v. 6, n. 4, 1981.
- [9] MICROSOFT. *How NTFS Works*. docs.microsoft.com/windows-server/storage/file-server/ntfs-overview
- [10] MATHUR, A. et al. The New ext4 Filesystem. *Proceedings of the Linux Symposium*, 2007.